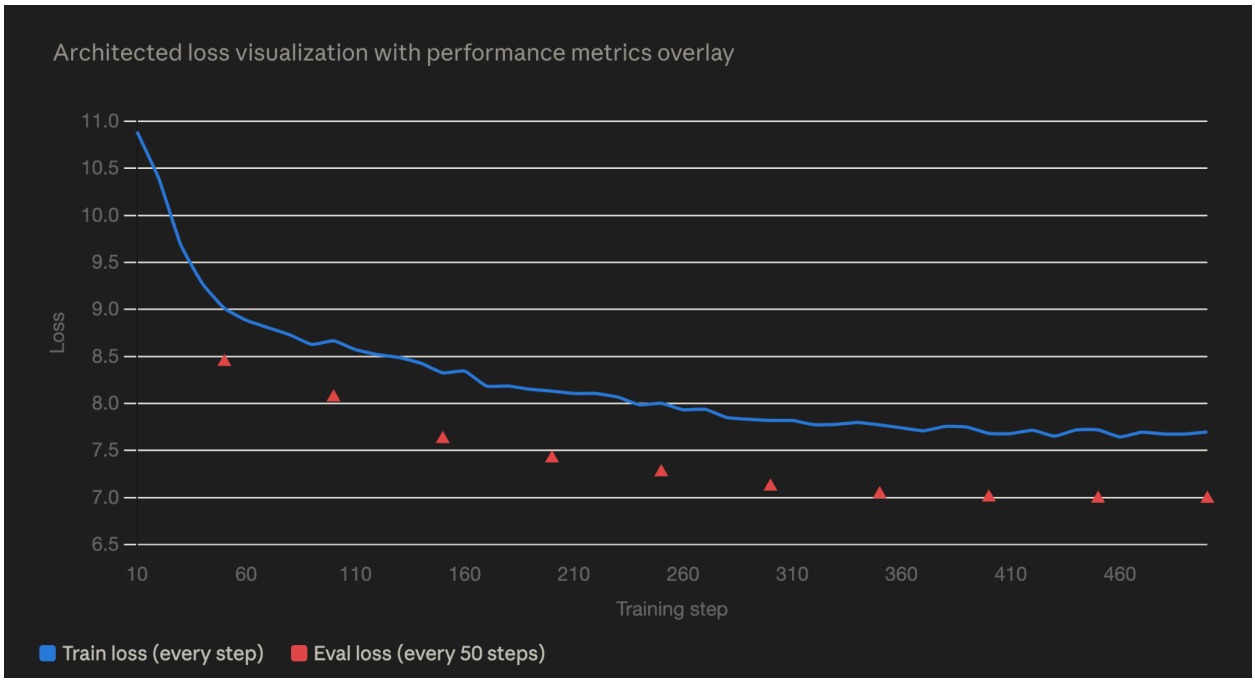
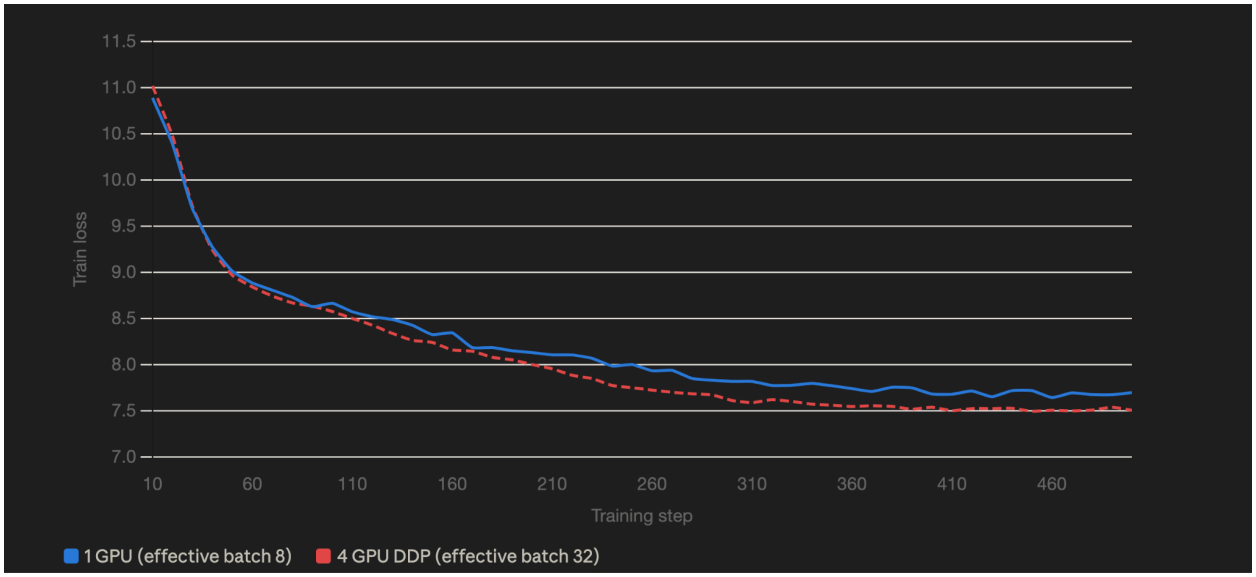
1. Maximum per device batch size: 16. I found it using some king of binary search. First 256 -> failed, then 32 -> failed, then 8 -> passed then 16 -> passed.
2. (nebius-ddp-training, pid=194855) [Comm] Trainable parameters: 774,030,080

3. **1-GPU run**:



Both curves drop fastest in the first ~150 steps then flatten out — train loss goes from 10.89 → 7.7 and eval loss from 8.44 → 6.99, with eval consistently below train (the eval set is easier/less noisy per-batch than the running train loss).
Wall-clock time: train_runtime: 362.5 s (about 6 min 2 s), matching the final progress bar 500/500 [06:02<00:00].
Throughput: train_samples_per_second: 22.07, train_steps_per_second: 1.379.
On the [Comm] lines: this run is world_size=1 (a single H100, SKYPILOT_NUM_GPUS_PER_NODE=1). NCCL still initializes (it always sets up a communicator, even a "ring" of one rank), but there's nothing to synchronize — gradients live on one device, so there's no all-reduce to perform. That's why every [Comm] line reads cumulative_comm_time_s=0.0000 and cumulative_comm_bytes=0, and the final summary shows ring all-reduce bytes/GPU/step: 0 bytes. The step-time hiccups you see every 10 steps (0.255s → ~1.12s) aren't communication overhead at all — they line up with the periodic eval passes (eval_runtime ~8.5-9s spread over the logging interval), not DDP sync cost. In other words, on this single-GPU config the "DDP" wrapper is a no-op for communication purposes; you'd only start seeing nonzero comm time/bytes once world_size > 1.

5. **4-GPU run**:



6.

Metric	1 GPU	4 GPU DDP
`train_runtime`	362.5 s	1424 s
`train_samples_per_second`	22.07	22.48
`train_steps_per_second`	1.379	0.351
Avg step time (from `[Comm]` lines)	0.560 s	2.819 s

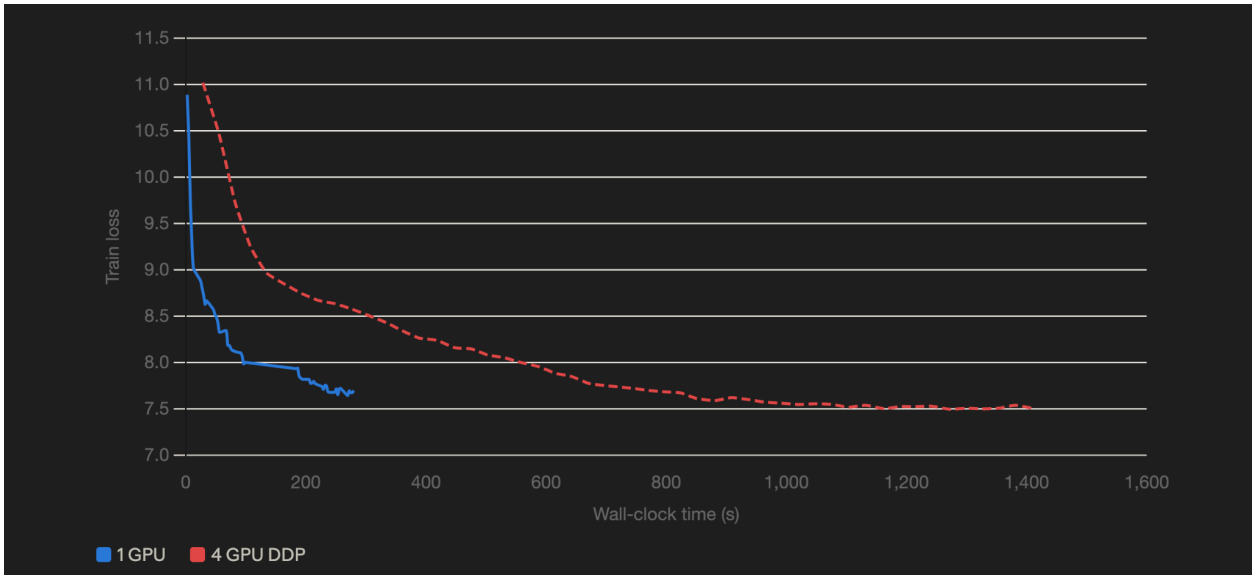
7.

Metric	1 GPU	4 GPU DDP
`eval_runtime`	8.5398 s	2.3158 s
`eval_samples_per_second`	58.432	215.474

Metric	1-GPU	4-GPU
Total measured comm time (s)	0.0000	1238.8968
Total measured comm bytes	0	1,548,060,160,000
Avg comm time per all-reduce (s)	0.000000	0.022777
Avg comm time per optimizer step (s)	0.000000	2.477794
Theoretical gradient payload per step (fp32)	3,096,120,320 bytes	3,096,120,320 bytes

Unlike training, final evaluation *does* scale well with more GPUs — eval runs each rank on its own shard of the eval set in parallel with no gradient sync needed (no backward pass, no all-reduce), so it's basically embarrassingly parallel. `eval_runtime` drops to about a quarter and `eval_samples_per_second` goes up ~3.7x, close to the theoretical 4x you'd hope for from 4 GPUs.

8.



9. DDP actually worsened training performance. It takes almost 4x time to train. We can see from the logs that the communication overhead takes up to 90% of the training. We can also see that the nodes don't use Infiniband and use regular TCP instead. This dramatically slows training down, since moving the gradients between nodes take a long time. On the other hand, we can see that inference is 4x times faster, since there's very low communication overhead at inference.

10. The best improvement would be to train over Infiniband or even better NVLink (use the 8 GPU clusters in nebius instead of the single gpu clusters). This would dramatically speed up the communication.